

ALMA MATER STUDIORUM · UNIVERSITY OF BOLOGNA

Master's Degree in Mathematics



**Constraint Programming Model
for the Shift-Design Personnel
Task Scheduling Problem**

Supervisor

Prof. Valentina Cacchiani

Co-supervisors

Dr. Marta Cavaleiro

Dr. Caterina Tamburini

Submitted by

Claudia Scarpa

0001166024

Academic Year 2025–2026

To every person I have ever met in my life:

*those who have been there from the beginning,
those who joined along the way and never left,
those who watch over my journey from above,
and those who walked beside me for a while.*

The person I am today is a linear combination of all of you.

Contents

1	Introduction	4
2	Theoretical Background	6
2.1	Introduction to Scheduling Problems	6
2.1.1	Personnel Scheduling Problems	7
2.1.2	The Shift-Design Personnel Task Scheduling Problem	9
2.2	Introduction to Constraint Programming	10
2.2.1	Basic Definitions	10
2.2.2	Constraint Satisfaction and Optimization Problem	11
2.2.3	Constraint Propagation and Tree Search	11
2.2.4	Global Constraints	13
2.2.5	The CP-SAT solver	13
2.3	Constraint Programming for Scheduling	14
3	Problem Description and MILP Model	16
3.1	The business problem at hand	16
3.2	MILP Mathematical Model	17
3.2.1	Sets and Parameters	17
3.2.2	Decision Variables	18
3.2.3	Constraints	19
3.2.4	Objective Function	21
3.3	Implementation by Optit	22
4	Constraint Programming Model	24
4.1	Data	25
4.1.1	Time	25
4.1.2	Activities	25
4.1.3	Employees	26
4.2	Decision variables	28
4.2.1	Primary variables	28
4.2.2	Auxiliary variables	29

4.3	Requirements	30
4.3.1	Consistency constraints	30
4.3.2	Legal Requirements	33
4.3.3	Company regulations	35
4.3.4	Special handling of the breaks	36
4.4	Objective function	37
4.5	Adaptation to fixed shifts	38
5	Lexicographic solution	40
5.1	Objective Functions and Idle Time	41
5.2	The Lexicographic Optimization Sequence	42
5.3	Implementation details: Warm Starting in CP-SAT	43
6	Computational results and comparison	44
6.1	Implementation of the CP Model	44
6.2	Experimental Setup	45
6.3	Results of the CP Model	46
6.3.1	Small Instance	46
6.3.2	Medium Instance	48
6.3.3	Big Instance	50
6.4	Comparison with the MILP Model	51
6.4.1	Small Instance	52
6.4.2	Medium Instance	52
6.4.3	Big Instance	53
6.5	Discussion	53
7	Conclusion	56
	Bibliography	58

Chapter 1

Introduction

Personnel scheduling is a fundamental planning activity in many organizations, including retail, healthcare, transportation, and service industries. The efficient allocation of employees to operational tasks has a direct impact on service quality, labor costs, and employee satisfaction. However, constructing high-quality schedules is often challenging due to the large number of operational, contractual, and legal constraints that must be simultaneously satisfied.

This thesis addresses a personnel scheduling problem arising from a real industrial application developed by Optit. More specifically, the focus is on the Shift-Design Personnel Task Scheduling Problem (SDPTSP), a class of workforce scheduling problems in which employee shifts are not predefined but must be generated as part of the optimization process. In this setting, shift design and task assignment are strongly interconnected decisions, making the problem particularly challenging from a computational perspective.

The company currently relies on a Mixed-Integer Linear Programming (MILP) formulation to generate workforce schedules. While MILP represents one of the most established approaches for combinatorial optimization, recent advances in Constraint Programming (CP) have demonstrated the effectiveness of CP-based methods for a wide range of scheduling applications. In particular, modern CP solvers provide specialized scheduling constructs and propagation mechanisms that can naturally capture many of the temporal and logical relationships arising in workforce scheduling problems.

The main objective of this thesis is therefore to investigate the suitability of Constraint Programming for the SDPTSP and to assess its performance on realistic industrial instances. To this end, a CP model based on Google's CP-SAT solver is developed and compared with the existing MILP approach currently adopted by the company.

The remainder of the thesis is organized as follows: Chapter 2 introduces the theoretical background required for the study, covering personnel scheduling problems, the Shift-Design Personnel Task Scheduling Problem, and the main concepts of Constraint Programming.

Chapter 3 presents the reference Mixed Integer Linear Programming formulation currently adopted in the industrial setting, describing its decision variables, constraints, and objective function.

Chapter 4 develops the main contribution of this work, namely a Constraint Programming formulation of the SDPTSP based on the CP-SAT solver. It presents the adopted modeling choices and emphasizes the key differences between this formulation and the MILP approach.

Chapter 5 describes the lexicographic optimization procedure used in the CP framework to address the multiple objectives arising in the scheduling problem.

Chapter 6 presents the computational experiments conducted on a set of realistic instances of different sizes. The performance of the proposed CP approach is analyzed and compared with that of the reference MILP formulation in terms of solution quality and computational efficiency.

Finally, Chapter 7 summarizes the main findings of the thesis, discusses the strengths and limitations of the proposed approach, and outlines possible directions for future research.

Chapter 2

Theoretical Background

This chapter provides the theoretical background needed to understand and contextualize the optimization models developed in the remainder of the thesis, serving as a conceptual bridge toward the practical application addressed in Chapters 3 and 4.

In the first part, the chapter outlines the fundamental concepts of scheduling problems [3], progressively narrowing the scope of investigation to Personnel Scheduling [7]. Particular attention will be devoted to the Shift-Design Personnel Task Scheduling Problem (SDPTSP), the core problem of this work, with reference to the academic studies that have formalized it in the literature [4] [2].

The second part of the chapter is devoted to Constraint Programming (CP). Drawing on key reference works on the subject [1] [3], it introduces the fundamental concepts of this declarative paradigm, including variables, domains, constraints, constraint propagation, search procedures, and global constraints. The chapter concludes by discussing the main features that make CP particularly suitable for scheduling applications, thereby providing the theoretical foundations for the CP model developed in Chapter 4.

2.1 Introduction to Scheduling Problems

Scheduling problems consist of allocating resources to tasks over given time periods. This is a very general definition and can be applied to a wide variety of problems. *Resources* may include machines in a workshop, gates at an airport, employees in an organization, or processing units in a computing environment, while *Tasks* may include operations in a production process, take-offs and landings at an airport, activities that must be performed, or the execution of computer programs.

From a computational perspective, scheduling problems are highly challenging. Most practical scheduling applications belong to the class of NP-hard combinatorial optimization problems, meaning that the computational effort required by exact solution methods may grow exponentially with the size of the instance. Consequently, obtaining

optimal solutions for large-scale real-world problems can become prohibitively expensive in terms of computation time. For this reason, the development of efficient and scalable solution approaches is of considerable theoretical and practical interest.

A widely adopted classification scheme for scheduling problems is the Graham notation $\alpha \mid \beta \mid \gamma$, which provides a compact description of the main characteristics of a scheduling problem:

- α describes the machine environment, including the number of machines, whether they are identical, unrelated, whether they operate in parallel, and other machine-specific characteristics such as processing speeds.
- β specifies the processing characteristics and technological constraints of the jobs, such as processing times, release dates, due dates, precedence relations, setup requirements, and other operational restrictions.
- γ represents the objective function to be optimized, for example minimizing the makespan, total completion time, lateness, tardiness, or other application-specific penalty measures.

Depending on the characteristics specified by these three fields, scheduling problems can be categorized into several classes, including flow shop, open shop, and job shop scheduling problems.

Although the Graham notation provides a useful framework for describing classical machine scheduling problems, its application to personnel scheduling is less straightforward. Unlike machines, employees are characterized by individual preferences, contractual agreements, legal regulations, and varying levels of qualification. As a result, personnel scheduling problems typically involve a richer set of constraints and objectives than those encountered in traditional manufacturing environments. For this reason, the personnel scheduling literature has developed dedicated classifications and modeling approaches to better capture the specific characteristics of workforce management problems.

2.1.1 Personnel Scheduling Problems

Personnel scheduling and rostering problems address the challenge of constructing employee schedules that satisfy workforce requirements while respecting legal regulations, contractual obligations, and organizational constraints.

These problems arise in a wide range of sectors, including healthcare, transportation, retail, manufacturing, and public services, where an appropriate allocation of personnel is essential to ensure efficient operations. In general, personnel scheduling seeks to answer two fundamental questions: how to organize the available workforce in a cost-effective manner and how to construct employee rosters that promote employee well-being and

satisfaction. Typical objectives include reducing labor costs, ensuring adequate service coverage, balancing workloads, and accommodating employee preferences whenever possible.

In practice, these goals are often conflicting. For example, a schedule that minimizes staffing costs may not adequately account for employee preferences, while a highly satisfactory roster for employees may require additional staffing resources. Balancing these competing objectives is one of the main challenges in personnel scheduling and rostering, and it explains the considerable academic and industrial interest in this class of problems.

Due to their complexity, personnel scheduling problems cannot always be described naturally within the classical Graham notation framework:

- the machine environment α is replaced by the workforce, whose size may vary over time and, in some cases, may even be treated as a decision variable. For example, the number of employees required on a particular day may depend on the expected demand and the desired service level.
- the field β must capture a much richer set of constraints than in traditional machine scheduling problems. In addition to operational requirements, personnel scheduling must account for contractual agreements, labor regulations, and organizational policies. Typical constraints include the maximum number of working hours per day or per week, limits on consecutive working days, minimum rest periods between shifts, and break requirements.
- the objective component γ also differs substantially from that of classical scheduling problems. Common objectives include minimizing labor costs, reducing uncovered demand, limiting the number of unassigned shifts, and improving fairness by balancing the workload among employees. In many practical applications, employee satisfaction and preference fulfillment are also important optimization criteria.

One of the earliest classification frameworks specifically developed for personnel scheduling problems was proposed by Baker. According to this taxonomy, personnel scheduling problems can be divided into three main categories:

- **Shift scheduling**, sometimes referred to as time-of-day scheduling, focuses on assigning employees within a single-day planning horizon. In its simplest form, the problem involves non-overlapping shifts, allowing staffing requirements for each shift to be considered independently. This setting is common in industrial environments, where fixed shift structures are frequently adopted. While such schedules are relatively straightforward to compute and implement, they become less effective when workforce demand varies significantly throughout the day. In

these situations, overlapping shifts are often required to provide a better match between staffing levels and demand patterns. Service-oriented environments such as call centers represent a typical example of this type of scheduling problem.

- **Days-off scheduling** or day-of-week scheduling, concerns the allocation of working and rest days over a weekly planning horizon. These problems arise when the operating schedule of an organization differs from the working schedule of its employees. A common example is an organization that operates seven days per week while employees work only five days per week. Additional constraints may also be imposed, such as requiring employees' days off to be consecutive.
- **Tour scheduling** combines both shift scheduling and days-off scheduling decisions. This type of problem is particularly relevant in organizations that operate continuously throughout the week and offer multiple shifts per day, such as hospitals, hotels, transportation services, and airlines. Since employees must be assigned both daily and weekly rest periods, the schedule must specify not only the days on which each employee works but also the corresponding shifts. As with the previous categories, the complexity of tour scheduling depends on several factors, including the planning horizon, the number of employees, and the applicable constraints. A particularly important factor is the length of the minimum planning interval, which may range from a few minutes to several hours depending on the application.

Within these categories, a further distinction concerns the definition of shifts. In many applications, shifts are predefined and employees must simply be assigned to them. In other settings, however, shifts are not known in advance and must be designed as part of the optimization process. This additional level of decision-making gives rise to shift design problems, in which workforce schedules are constructed according to the characteristics of the activities to be performed and the underlying demand requirements.

2.1.2 The Shift-Design Personnel Task Scheduling Problem

The Shift-Design Personnel Task Scheduling Problem (SDPTSP) is a class of personnel scheduling problems in which work shifts are not predefined but must be generated as part of the optimization process.

The SDPTSP integrates two decision layers that are often treated separately: shift design and personnel assignment. Given a set of employees and activities, the objective is to determine both the structure of the work shifts and the allocation of employees to activities while satisfying operational requirements and workforce constraints.

The simultaneous optimization of these decisions significantly increases the complexity of the problem, as shift-design choices directly affect the feasibility of activity assignments and vice versa. For this reason, the SDPTSP has been the subject of several

studies in the scheduling literature, including the formulations presented in [4] and [2], which provide the basis for the models developed in this thesis.

2.2 Introduction to Constraint Programming

Constraint Programming (CP) is a declarative paradigm for modeling and solving combinatorial problems. A problem is described through a set of decision variables, their associated domains, a collection of constraints, and, when appropriate, an objective function. Rather than specifying the sequence of operations required to construct a solution, the model defines the properties that feasible solutions must satisfy. The task of finding feasible or optimal solutions is then delegated to a specialized solver, which automatically combines constraint propagation and search techniques to explore the solution space efficiently.

CP has proven particularly effective for problems characterized by complex combinatorial structures and numerous logical relationships among decision variables. Its expressive modeling framework allows constraints to be represented in a natural and intuitive way, often leading to formulations that closely reflect the underlying problem structure.

2.2.1 Basic Definitions

The fundamental components of a constraint programming model are variables, domains, and constraints.

- **Variables and Domains:** The domain of a variable is the set of values that can be assigned to it. They are typically finite, although infinite discrete domains may also be considered. The domain associated with a variable x is denoted by $\mathcal{D}(x)$. Let $V = \{x_1, \dots, x_n\}$ be a set of variables and let $D = \{\mathcal{D}(x_1), \dots, \mathcal{D}(x_n)\}$ denote the corresponding set of domains.

An assignment $a = (a_1, \dots, a_n)$ is an element of the Cartesian product $\mathcal{D}(x_1) \times \dots \times \mathcal{D}(x_n)$. In other words, an assignment associates each variable x_i with a value $a_i \in \mathcal{D}(x_i)$.

- **Constraints:** A constraint c over the domains D is a function

$$c : \mathcal{D}(x_{i_1}) \times \dots \times \mathcal{D}(x_{i_k}) \rightarrow \{\text{true}, \text{false}\},$$

where $V' = \{x_{i_1}, \dots, x_{i_k}\} \subseteq V$ is the subset of variables involved in the constraint. The cardinality $|V'|$ is called the *arity* of the constraint. Constraints of arity one and two are referred to as unary and binary constraints, respectively.

An assignment $a \in \mathcal{D}(x_1) \times \dots \times \mathcal{D}(x_n)$ satisfies the constraint c if and only if

$$c(a_{i_1}, \dots, a_{i_k}) = \text{true}.$$

2.2.2 Constraint Satisfaction and Optimization Problem

Many search and scheduling problems can be formulated as instances of either the Constraint Satisfaction Problem (CSP) or the Constraint Optimization Problem (COP). These two frameworks provide the theoretical foundation for most CP models and constitute the basis of modern CP solvers.

- **Constraint Satisfaction Problem (CSP):** A CSP can be informally described as follows: given a set of variables, their domains, and a collection of constraints, find an assignment of values to the variables such that all constraints are satisfied.

Formally, an instance I of a CSP is defined by the tuple

$$I = (V, D, C),$$

where V is a finite set of variables, D is the set of associated domains, and C is a finite set of constraints. An assignment a is feasible, and therefore a solution of I , if and only if it satisfies every constraint in C . The objective is either to find a feasible solution or to prove that no such solution exists.

- **Constraint Optimization Problem (COP):** Unlike a CSP, a Constraint Optimization Problem seeks a feasible solution that optimizes a given objective function. Formally, an instance I of a COP is defined by the tuple

$$I = (V, D, C, z),$$

where (V, D, C) defines a CSP and z is an objective function

$$z : \mathcal{D}(x_1) \times \cdots \times \mathcal{D}(x_n) \rightarrow \mathbb{R}.$$

An assignment a is an optimal solution of I if it is feasible and optimizes the objective function z , either by minimization or maximization.

Both CSPs and COPs are computationally challenging problems. In general, determining whether a CSP admits a feasible solution is NP-complete, while finding an optimal solution to a COP is typically NP-hard.

2.2.3 Constraint Propagation and Tree Search

Since variable domains are generally finite, a CSP can in principle be solved using a generate-and-test procedure that enumerates all possible assignments and checks whether they satisfy the constraints. However, this approach quickly becomes impractical, as the size of the search space grows exponentially with the number of variables. In the worst case, every possible assignment must be examined before a solution is found or infeasibility is proven.

To address this combinatorial explosion, CP solvers rely on a combination of **constraint propagation** and systematic **tree search**.

The purpose of constraint propagation is to reduce the search space by identifying and removing values that cannot belong to any feasible solution. By repeatedly analyzing the relationships between variables, domains, and constraints, propagation algorithms infer additional information from the model and eliminate inconsistent values before or during the search process. As a result, many infeasible regions of the search space can be discarded without explicit enumeration.

The goal of constraint propagation is to remove values that cannot participate in any feasible solution while preserving all feasible solutions of the original problem. In practice, this is achieved by enforcing consistency properties on variables and constraints. Intuitively, a consistency property specifies conditions under which values in the domains are considered compatible with the constraints of the problem.

Since enforcing complete global consistency is generally computationally intractable, CP solvers typically enforce weaker forms of local consistency through specialized filtering algorithms.

- **k-Consistency:** A CSP is said to be k -consistent if any consistent assignment to $k - 1$ variables can be extended to any k -th variable while preserving consistency with the constraints. Important special cases include node consistency and arc consistency.
- **Domain Consistency:** Also known as generalized arc consistency for non-binary constraints, domain consistency requires that every value remaining in a variable domain participates in at least one satisfying assignment of the associated constraint.
- **Bounds Consistency:** When domains are ordered, such as integer domains, consistency is enforced only on the lower and upper bounds of each domain. This typically provides weaker filtering than domain consistency but can be achieved at a significantly lower computational cost.

Because the removal of values from one domain may create new opportunities for filtering in related constraints, propagation algorithms are applied repeatedly until no further reductions are possible. This state is referred to as a fixed point of the propagation process.

Constraint propagation alone is generally insufficient to solve a CSP completely. Therefore, it is combined with systematic search procedures that progressively assign values to variables and explore the resulting search tree. In optimization problems, search is often integrated with branch-and-bound techniques, which use objective bounds to prune regions of the search space that cannot contain better solutions.

2.2.4 Global Constraints

A **global constraint** represents a structured set of constraints as a single entity. For example, the `alldifferent` constraint requires that a set of variables take distinct values. From a modeling perspective, it represents a set of pairwise disequations, but operationally it allows the solver to apply specialized graph-theory propagation techniques that can detect infeasibility earlier than considering each pair individually.

To solve a CSP featuring these global entities efficiently, CP systems enforce generalized forms of local consistency:

- A constraint $c(x_1, \dots, x_n)$ is **generalized arc consistent (GAC)** if for every variable x_i and every value $v_i \in \mathcal{D}(x_i)$, there exist compatible values for the other variables such that the constraint is satisfied.

In scheduling problems, global constraints are exceptionally useful. They can compactly model conditions such as no overlapping assignments, skill coverage, or resource limits, and their global propagation view can prune the search tree significantly.

2.2.5 The CP-SAT solver

Modern Constraint Programming systems often combine classical CP techniques with methods originating from Boolean satisfiability (SAT) and Integer Programming. One of the most successful examples is the CP-SAT solver developed by Google and distributed through the OR-Tools optimization suite.

CP-SAT models problems through integer and boolean decision variables and relies on a hybrid solution strategy combining constraint propagation, SAT-based conflict learning, and branch-and-bound search. Unlike traditional CP solvers, which are primarily based on domain propagation, CP-SAT also exploits techniques inherited from modern SAT solvers, such as clause learning and conflict-driven search, allowing it to handle large combinatorial optimization problems effectively.

Another important feature of CP-SAT is its support for specialized scheduling constructs, including interval variables, optional interval variables, and global constraints such as `NoOverlap`. These abstractions provide a natural way to model temporal scheduling decisions while benefiting from dedicated propagation algorithms.

The Constraint Programming model developed in this thesis is implemented using CP-SAT through the OR-Tools framework. Several modeling choices adopted in the following chapters, including the use of interval variables and global scheduling constraints, directly exploit features provided by this solver.

2.3 Constraint Programming for Scheduling

The characteristics of Constraint Programming make it particularly well suited for scheduling applications, including the Shift-Design Personnel Task Scheduling Problem considered in this thesis. In such problems, employees, shifts, and activities can be modeled through decision variables whose domains represent feasible assignments. Constraints are then used to express operational requirements, such as staffing coverage, employee availability, workload limitations, rest-period regulations, and skill compatibility requirements.

A major advantage of CP lies in its ability to represent complex logical and combinatorial relationships in a natural and expressive way. Through the use of specialized propagation algorithms and global constraints, CP solvers can eliminate large portions of the search space before explicit enumeration becomes necessary, significantly improving computational efficiency.

By combining a declarative modeling framework with powerful propagation and search mechanisms, CP offers both flexibility in problem representation and effectiveness in solution search. These features have made CP a widely adopted approach for a variety of scheduling and rostering applications. Furthermore, modern optimization frameworks often integrate CP with complementary techniques, such as Mixed-Integer Linear Programming (MILP) relaxations or decomposition methods, in order to exploit the strengths of different paradigms and tackle increasingly complex scheduling problems.

Chapter 3

Problem Description and MILP Model

This chapter presents a detailed description of the business problem under study, progressing from a general overview to a more formal and structured representation. To formally describe the problem, we introduce a mathematical model by reporting the Mixed-Integer Linear Programming (MILP) formulation proposed by Gusmeroli and Bettinelli [2], which constitutes one of the first explicit formulations of the Shift-Design Personnel Task Scheduling Problem (SDPTSP) and is used here purely as a benchmark reference. Finally, we refer to the solution approach developed by Optit, which is also used as a benchmark in the subsequent analysis, although only limited information about its internal structure is available.

3.1 The business problem at hand

The problem under consideration is classified as a Shift-Design Personnel Task Scheduling Problem (SDPTSP), a complex variant of personnel scheduling that integrates multiple decision layers within a unified framework. Unlike classical rostering approaches, where shifts are predefined and employees are simply assigned to them, this problem requires the simultaneous optimization of several interdependent components.

In particular, three main dimensions must be addressed jointly. The first is shift design, which determines the start and end times of each employee's working day. The second is task assignment, which allocates activities such as checkout operations, shelf stocking, or customer assistance within each shift to satisfy time-varying demand. The third dimension concerns staff heterogeneity, reflecting the fact that employees differ in skills, qualifications, and availability, and therefore cannot be considered interchangeable.

In practice, the problem is generated from operational data provided by the company. These inputs include forecasted demand for each activity over time, employee availability calendars, qualification matrices specifying which activities each employee is allowed to perform, contractual and legal working limits, and company-specific op-

erational policies. Based on these data, the optimization model determines both the structure of employee shifts and the assignment of activities throughout the planning horizon.

The primary objective of the problem is to minimize uncovered demand, i.e., the portion of required work not assigned to any employee. In addition, the model incorporates employee preferences and suitability within the objective function, favoring assignments that better match individual skills and experience. This not only ensures demand satisfaction but also improves the overall quality of the schedule. At the same time, any feasible solution must satisfy a complex set of constraints, including legal requirements (such as limits on working hours and mandatory rest periods) as well as company-specific rules designed to ensure operational feasibility.

3.2 MILP Mathematical Model

The SDPTSP model implemented in Optit is formulated as a Mixed Integer Linear Program (MILP) and is reported here as a benchmark reference. It provides a unified representation of the problem, simultaneously capturing shift design and task assignment within a single optimization framework. The formulation incorporates the main structural components of the problem, including time dependent demand, employee availability, and skill compatibility, while enforcing the relevant legal and organizational constraints. No modifications or extensions are introduced with respect to the original formulation, which is adopted in its original form as presented in the literature.

3.2.1 Sets and Parameters

To formally describe the problem, we introduce the main sets and parameters used in the MILP formulation.

Let R , A , D , and T denote the sets of employees, activities, days, and timeslots, respectively. The planning horizon is discretized into uniform time intervals, allowing the model to capture variations in demand throughout the day. Each day $d \in D$ is partitioned into the set of timeslots T , so that time is represented by the pair (t, d) , where $t \in T$ identifies the timeslot within day d . This structure allows the model to capture intraday dynamics while preserving the distinction between different days.

The parameter ts represents the length of each timeslot (in minutes), while tsD denotes the total number of timeslots in a day. Demand is modeled through the function $dem(a, d, t_1, t_2)$, which specifies the amount of work required for activity a on day d within a given time interval $[t_1, t_2]$.

Employee related information is captured through two main parameters. The compatibility matrix $cRA(r, a)$ is a binary parameter indicating whether an employee is qualified

to perform a given activity, while the availability matrix $cRTD(r, t, d)$ is a binary parameter specifying whether an employee is available at a given timeslot and day.

In addition, the model includes a penalty parameter $p(a)$ associated with unmet demand of a certain activity, reflecting the relative importance of different activities.

The parameter $c(r, a)$ represents the quality of the assignment between employee r and activity a . Notably, lower values correspond to better matches, with near zero values representing highly suitable assignments. This is consistent with the minimization nature of the objective function, which naturally favors lower cost assignments.

Finally, a sufficiently large constant M is introduced to enforce logical relationships between decision variables within the model.

For notational convenience, we introduce subsets of the planning horizon to ensure that all time indexed expressions are well defined. For the set of days D , we denote by D_k the first k days, and by $D_{\bar{k}} = D \setminus D_k$ the set of remaining days. Similarly, D^k denotes the last k days, and $D^{\bar{k}}$ the set of all days except the last k . Analogous definitions apply to the set of time slots T , yielding the subsets T_k , $T_{\bar{k}}$, T^k , and $T^{\bar{k}}$. These definitions ensure that all interval based expressions of the form $[t, t + k)$ or $[d, d + k]$ are well defined within the model.

3.2.2 Decision Variables

The MILP formulation uses a set of decision variables to represent scheduling decisions and deviations from demand satisfaction. The primary decision variable of the model is $x(r, a, t, d)$, which directly encodes the assignment of employees to activities over time-while additional variables are introduced to capture structural properties of the schedule and facilitate the formulation of constraints.

The full set of variables is defined as follows:

- $x(r, a, t, d) \in \{0, 1\}$: indicates whether employee r is assigned to activity a at timeslot t on day d . This is the primary decision variable of the model, as it fully determines the assignment structure.
- $y(r, a, t, d) \in \{-1, 0, 1\}$: represents changes in work status for employee r on activity a at timeslot t of day d , capturing start, stop, and inactivity transitions, respectively.
- $z(r, d) \in \{0, 1\}$: indicates whether employee r works on day d , linking detailed assignments to daily level constraints.
- $b(r, d) \in T$, $e(r, d) \in T$: represent the start and end of the working period for employee r on day d , derived from the assignment variables.

- $\alpha(a, d, t_1, t_2) \geq 0$: represents the unmet demand for activity a on day d within the time interval $[t_1, t_2]$.

By construction, all variables other than $x(r, a, t, d)$ are auxiliary and serve to express structural properties of the schedule or to facilitate the formulation of constraints. As a result, the entire scheduling decision is ultimately driven by the assignment variable $x(r, a, t, d)$.

3.2.3 Constraints

Problem Defining Constraints (D)

These constraints define the internal structure of the model by linking decision variables and ensuring consistency.

The first group of constraints links assignment and transition variables, ensuring that the variable $y(r, a, t, d)$ correctly captures changes in activity assignment between consecutive timeslots:

$$y(r, a, t, d) = x(r, a, t, d) \quad \forall r \in R, a \in A, t \in T_1, d \in D, \quad (3.1)$$

$$y(r, a, t, d) = x(r, a, t, d) - x(r, a, t-1, d) \quad \forall r \in R, a \in A, t \in T_{\bar{1}}, d \in D. \quad (3.2)$$

The following constraints define whether an employee works on a given day by linking the assignment variables $x(r, a, t, d)$ to the daily indicator $z(r, d)$.

$$\sum_{a \in A} \sum_{t \in T} x(r, a, t, d) \leq M \cdot z(r, d) \quad \forall r \in R, d \in D, \quad (3.3)$$

$$\sum_{a \in A} \sum_{t \in T} x(r, a, t, d) \geq z(r, d) \quad \forall r \in R, d \in D, \quad (3.4)$$

The start and end of the daily working interval for employee r on day d is defined by the variables $b(r, d)$ and $e(r, d)$, which are derived from the assignment variables and identify, respectively, the first and last timeslot in which the employee is working.

$$b(r, d) \leq t + tsD \left(1 - \sum_{a \in A} x(r, a, t, d) \right) \quad \forall r \in R, t \in T, d \in D, \quad (3.5)$$

$$e(r, d) \geq (t+1) \sum_{a \in A} x(r, a, t, d) \quad \forall r \in R, t \in T, d \in D. \quad (3.6)$$

Finally, basic feasibility conditions on the assignment variables are enforced. In particular, these constraints ensure that each employee is assigned to at most one activity at any given timeslot and that the working interval is well defined (i.e., the start time does not exceed the end time).

$$\sum_{a \in A} x(r, a, t, d) \leq 1 \quad \forall r \in R, t \in T, d \in D, \quad (3.7)$$

$$b(r, d) \leq e(r, d) \quad \forall r \in R, d \in D. \quad (3.8)$$

Legal Constraints (L)

This group of constraints models regulatory requirements, ensuring compliance with labor laws regarding working time and rest periods. The parameters k_{Li} denote regulatory limits, which may depend on the employee r (e.g., due to different contracts or roles), and are therefore not necessarily identical across all employees.

The first constraints impose limits on the total amount of work assigned to each employee each day and each week, respectively:

$$\sum_{a \in A} \sum_{t \in T} x(r, a, t, d) \leq k_{L1,r} \quad \forall r \in R, d \in D, \quad (3.9)$$

$$\sum_{a \in A} \sum_{t \in T} \sum_{d \in D} x(r, a, t, d) \leq k_{L2,r} \quad \forall r \in R. \quad (3.10)$$

The following constraint limits the number of consecutive working days

$$\sum_{d' \in [d, d+k_{L3,r}]} z(r, d') \leq k_{L3} \quad \forall r \in R, d \in \overline{D^{k_{L3}}}. \quad (3.11)$$

Break-related constraints within a working day are enforced by limiting the maximum amount of work that can be assigned within any time window of length $k_{L4} + j_{L4}$, where j_{L4} denotes the minimum required break duration. This ensures that employees cannot work continuously for more than k_{L4} timeslots without a break.

$$\sum_{t' \in [t, t+k_{L4}+j_{L4}]} \sum_{a \in A} x(r, a, t', d) \leq k_{L4} \quad \forall r \in R, t \in T^{k_{L4}+j_{L4}-1}, d \in D. \quad (3.12)$$

The duration of each working day is regulated by limiting the time span between its start and end:

$$e(r, d) - b(r, d) \leq k_{L5,r} \quad \forall r \in R, d \in D. \quad (3.13)$$

Finally, minimum rest between consecutive working days is enforced through:

$$tsD + b(r, d) - e(r, d-1) \geq k_{L6,r} \quad \forall r \in R, d \in D_{\bar{1}}. \quad (3.14)$$

Together, these constraints ensure compliance with legal requirements regarding workload, rest periods, and working conditions.

Company Constraints (G)

These constraints encode company specific operational rules aimed at improving schedule quality. The parameters k_{Gi} represent company defined operational limits and policy

parameters regulating schedule structure and service quality. The following constraint ensures a minimum amount of work after a change of activity:

$$\sum_{t' \in [t, t+k_{G1})} \sum_{a \in A} x(r, a, t', d) \geq \sum_{a \in A} y(r, a, t, d) \cdot k_{G1} \quad \forall r \in R, t \in T^{k_{G1}-1}, d \in D. \quad (3.15)$$

Demand satisfaction is enforced through

$$\sum_{t \in [t_1, t_2)} \sum_{r \in R} x(r, a, t, d) \cdot ts + \alpha(a, d, t_1, t_2) \geq dem(a, d, t_1, t_2) \quad \forall a \in A, d \in D, t_1, t_2 \in T. \quad (3.16)$$

where $\alpha(a, d, t_1, t_2)$ represents the amount of unmet demand and is included as a decision variable in the objective function.

To avoid fragmented schedules, the model enforces a minimum duration for each activity:

$$\sum_{t' \in [t, t+k_{G3,a})} x(r, a, t', d) \geq y(r, a, t, d) \cdot k_{G3,a} \quad \forall r \in R, a \in A, t \in T^{k_{G3,a}-1}, d \in D. \quad (3.17)$$

Finally, feasibility with respect to employee–activity compatibility and temporal availability is ensured by

$$x(r, a, t, d) \leq cRA(r, a) \quad \forall r \in R, a \in A, t \in T, d \in D, \quad (3.18)$$

$$x(r, a, t, d) \leq cRTD(r, t, d) \quad \forall r \in R, a \in A, t \in T, d \in D. \quad (3.19)$$

Checkout Management (G6)

The formulation proposed by Gusmeroli and Bettinelli includes an additional subset of constraints modeling checkout sequencing rules. These constraints are not included in the formulation presented in this work, as they are highly domain-specific and fall outside the scope of our analysis.

3.2.4 Objective Function

The objective function balances demand satisfaction and assignment quality:

$$\min \left(\sum_{a \in A} \sum_{d \in D} \sum_{t_1, t_2 \in T} p(a) \cdot \alpha(a, d, t_1, t_2) + \sum_{r \in R} \sum_{a \in A} \sum_{t \in T} \sum_{d \in D} x(r, a, t, d) \cdot c(r, a) \right) \quad (3.20)$$

The first term penalizes unmet demand, while the second promotes high quality assignments. The resulting formulation enforces a tradeoff between operational efficiency and assignment suitability.

3.3 Implementation by Optit

The model is implemented by Optit using Mosel and Java in a modular architecture separating optimization and data management. Mosel is the modeling and programming language of the FICO Xpress optimization suite, designed to provide a high level algebraic environment for defining complex mathematical models. In this architecture, Mosel is used to encode the MILP formulation, closely mirroring its mathematical structure and allowing for efficient interaction with the Xpress solver. Java handles preprocessing and postprocessing, including data validation, transformation into model inputs, and interpretation of solutions. This separation improves flexibility and facilitates adaptation to different problem instances while maintaining a robust and extensible implementation.

Chapter 4

Constraint Programming Model

This chapter presents the Constraint Programming formulation of the employee scheduling problem addressed in this thesis, which constitutes the core contribution of this work. It provides a detailed description of the input data, decision variables, constraints, and objective function, together with selected insights on the corresponding CP-SAT implementation.

Two modeling approaches were considered during the development of the model. The first one is based on a predefined set of fixed shifts, while the second, and main, formulation relies on flexible shifts, where the working periods are implicitly determined by the assigned activities.

Although the fixed-shift model was developed first, due to its simpler structure and easier validation, it is only briefly discussed at the end of the chapter. Most of its components naturally extend to the flexible-shift formulation, which provides a more accurate representation of the real-world problem by allowing the generation of personalized schedules for each employee, rather than restricting decisions to a predefined set of shifts. For this reason, the chapter primarily focuses on the flexible-shift formulation, emphasizing the key modeling choices and highlighting the aspects in which CP differs from the MILP approach introduced earlier.

Although the CP formulation addresses the same underlying scheduling problem introduced in Chapter 3, some modeling choices differ from the MILP formulation due to the interval-based structure of CP-SAT. In particular, assignment and temporal positioning are represented separately, activities are preprocessed into indivisible subactivities, and optimization is performed lexicographically rather than through a weighted-sum objective. Another key modeling difference concerns the treatment of breaks. In the MILP formulation, break requirements are enforced based on the amount of effective working time, i.e., the time spent performing assigned activities. In contrast, in the CP formulation breaks are modeled with respect to the elapsed time within the shift span, independently of whether the employee is actively working or idle. This leads to a different but related representation of rest requirements, while preserving the same

underlying operational constraints.

These differences primarily concern the mathematical formulation and solution strategy, although some operational aspects, such as break management, are modeled differently.

4.1 Data

The input data are discussed and explained in the following subsections and then summarized in Table 4.1 for readability and ease of reference.

4.1.1 Time

Time is discretized into a finite set of timeslots $\mathcal{T} = \{0, \dots, T\}$ over a planning horizon $\mathcal{D} = \{0, \dots, D\}$ (one week in our case). Each day is partitioned into a fixed number of equally spaced timeslots; specifically for our problem, we consider 96 timeslots per day, corresponding to intervals of 15 minutes.

A key modeling choice is that each timeslot represents a specific point in time rather than a time interval. For example, a timeslot index $t = 32$ corresponds to a precise time of the day (e.g., 08:00 of day 1), while a duration of two timeslots represents a time span of 30 minutes.

This distinction is important for correctly interpreting start and end variables. Activity durations are therefore expressed as differences between timeslot indices, ensuring consistency between point-based representation and interval lengths.

The global time index $t \in \mathcal{T}$ spans the entire planning horizon. The corresponding day can be derived as:

$$day(t) = \left\lfloor \frac{t}{|\mathcal{T}_{day}|} \right\rfloor,$$

where $|\mathcal{T}_{day}|$ denotes the number of timeslots per day.

This unified time representation simplifies the formulation of temporal constraints and avoids the need for separate indexing over days and times.

4.1.2 Activities

Let $\mathcal{A} = \{0, \dots, A\}$ denote the set of all the activities that have to be scheduled. Each activity $a \in \mathcal{A}$ is characterized by the following attributes:

- $id(a)$: unique identifier used to distinguish activities and to facilitate preprocessing operations.
- $dem(a)$: required duration of the activity. During the preprocessing phase, this demand is converted from minutes into the corresponding number of timeslots.

- $day(a)$: day of the planning horizon in which the activity must be performed.
- $[t_1(a), t_2(a)] \subset \mathcal{T}$: time window within which the demand of the activity must be scheduled. These values represent, respectively, the earliest possible starting timeslot for the activity and the latest possible ending one.
- $k_{G3}(a)$: minimum consecutive amount of time that must be spent on the activity once an employee starts working on it. This value is also converted from minutes into a number of timeslots during the preprocessing phase.
- $p(a)$: penalty incurred for each timeslot of unmet demand.
- a set of skills required to perform the activity.

4.1.3 Employees

Each employee $r \in \mathcal{R} = \{0, \dots, R\}$ is characterized by a set of features and parameters describing their abilities, availability, legal constraints, historical information, and preferences. All time-related parameters are expressed in timeslots unless otherwise specified.

- $id(r)$: unique identifier for the employee.
- $k_{L1}(r)$: maximum daily working time. Here, we consider only the actual time spent working, so breaks and idle periods within the schedule are not included.
- $k_{L2}(r)$: maximum weekly working time, defined analogously to the daily limit but over the entire week.
- $k_{L3}(r)$: maximum number of consecutive working days.
- $k_{L5}(r)$: maximum daily span. In contrast to $k_{L1}(r)$, this includes all the time spent at work, including breaks and idle periods. It is computed as the length of the shift.
- $k_{L6}(r)$: minimum required rest time between consecutive shifts.
- $k_{L7}(r)$: maximum number of working days within the planning horizon.
- $\sigma(r, d)$ and $\varepsilon(r, d)$ denote the first and last available times, respectively, on day d .

In addition, we consider historical data that must be taken into account:

- $h(r)$: last working timeslot before the beginning of the planning horizon.
- $w(r)$: the number of consecutive days worked since the last day off.

Each employee is also associated with:

- a set of skills; during the preprocessing phase, this information is used to derive $\mathcal{A}^r \subset \mathcal{A}$, the set of activities that the employee is qualified to perform.
- a preference score $c(r, a)$ indicating how much the employee dislikes a given activity. The scores are defined so that higher values correspond to lower preference, making them consistent with a minimization objective.

Table 4.1: Summary of input data

Data	Definition
$\mathcal{R} = \{0, \dots, R\}$	set of employees
$\mathcal{D} = \{0, \dots, D\}$	set of days
$\mathcal{T} = \{0, \dots, T\}$	set of timeslots
$\mathcal{A} = \{0, \dots, A\}$	set of activities
For every activity $a \in \mathcal{A}$:	
$t_1(a) \in \mathcal{T}$	minimum possible starting timeslot of activity a
$t_2(a) \in \mathcal{T}$	maximum possible ending timeslot of activity a
$dem(a)$	demand of timeslots for activity a
$k_{G3}(a)$	minimum consecutive timeslots on activity a
$p(a)$	penalty associated with each unmet timeslot of activity a
For every employee $r \in \mathcal{R}$:	
$\mathcal{A}^r \subset \mathcal{A}$	set of activities that employee r is qualified to do
$k_{L1}(r)$	maximum daily timeslots
$k_{L2}(r)$	maximum weekly timeslots
$k_{L3}(r)$	maximum consecutive working days
$k_{L5}(r)$	maximum daily span
$k_{L6}(r)$	minimum rest between shifts
$k_{L7}(r)$	maximum working days in the planning horizon
$h(r) \in \mathcal{T}$	last worked hour (before the start of the planning horizon)
$w(r)$	number of worked days since the previous day-off
For every employee $r \in \mathcal{R}$, $\forall d \in \mathcal{D}$:	
$\sigma(r, d) \in \mathcal{T}$	first available timeslot for employee r on day d
$\varepsilon(r, d) \in \mathcal{T}$	last available timeslot for employee r on day d
For every employee $r \in \mathcal{R}$, $\forall a \in \mathcal{A}$:	
$c(r, a)$	preference score of employee r for activity a

4.2 Decision variables

In this section, we introduce the decision variables used in the CP formulation. For clarity, a complete overview is provided in Table 4.2, while the variables are discussed in detail in the following subsections.

The variables are divided into two groups. The primary variables represent the core decisions of the model, namely the assignment of activities and their scheduling in time. The auxiliary variables are instead derived from the primary ones and are introduced to simplify the formulation of the constraints and improve model readability.

In addition to the mathematical definition, we also provide insights into their implementation in the CP-SAT framework, focusing in particular on constructs that have no direct counterpart in linear programming, such as interval variables and logical constraints.

4.2.1 Primary variables

The model is primarily driven by three sets of decision variables, referred to as primary variables, as they represent the core decisions that the optimization model must determine:

- $x_{r,a} \in \{0, 1\}$

This is the main assignment variable and indicates if activity a is assigned to employee r . In the MILP formulation, assignment and temporal positioning are encoded jointly through the indexed variable $x_{r,a,t,d}$, while in the CP formulation these two aspects are separated: assignment is represented by $x_{r,a}$, whereas temporal positioning is modeled through start/end variables and interval variables.

- $S_a, E_a \in [t_1(a), t_2(a)]$

These variables represent the starting and ending timeslots of activity $a \in \mathcal{A}$, respectively. They define the temporal placement of the activity within the planning horizon, ensuring that each activity is scheduled within its admissible time window.

To link temporal decisions with assignment choices, we exploit CP-SAT *interval variables*. An interval variable models a task through a start time, a duration, and an end time, and it can optionally be active depending on a Boolean condition. In this formulation, each activity is associated with an interval that is active only if the activity a is assigned to employee r .

This is implemented as follows in the CP-SAT model:

```
activity_interval[r,a] = model.new_optional_interval_var(  
    start = start[a],  
    size = duration[a],
```

```

    end = end[a],
    is_present = assignment[r,a]
)

```

Code Snippet 4.1: Definition of optional interval variables in CP-SAT (OR-TOOLS). The Boolean variable `is_present` corresponds to $x_{r,a}$ and determines whether the interval associated with activity a assigned to employee r is active.

This construction ensures that the temporal variables associated with an activity are taken into account only when the activity is effectively assigned to the employee. As a result, assignment and scheduling decisions are tightly integrated within the same modeling framework.

Observation: In the CP-SAT framework, interval variables represent continuous time intervals. As a consequence, a single interval cannot be interrupted once it has started. This creates a modeling challenge when an employee needs to temporarily stop working on an activity (e.g., due to a break) and then resume it later. To address this limitation, each activity is decomposed during preprocessing into smaller sub-activities. The duration of each sub-activity is chosen to be equal to the minimum consecutive working time $k_{G3}(a)$. This ensures that each sub-activity satisfies the minimum processing requirement while allowing flexibility in the schedule. As a result, the model can assign different sub-activities of the same original task at different times, possibly separated by breaks or idle periods. In this setting, the duration of each activity becomes fixed, and the end time can be expressed as $E_a = S_a + k_{G3}(a)$. Nevertheless, for the sake of generality and clarity of exposition, we retain both S_a and E_a as decision variables in the mathematical formulation. This preprocessing step transforms the original demand coverage formulation into a scheduling problem over indivisible activity blocks, which is more compatible with the interval based structure of CP-SAT.

4.2.2 Auxiliary variables

We also introduce a set of secondary (auxiliary) variables; these variables are not independent decisions but are instead derived from the primary variables to facilitate modeling and constraint definition.

- $z_{r,d} \in \{0, 1\}$
This binary variable indicates whether employee r is working on day d . It captures the daily working status of each employee and requires consistency constraints to link it with the activity assignment variables.
- $b_{r,d}, e_{r,d} \in [\sigma(r, d), \varepsilon(r, d)]$
These variables represent, respectively, the start and end times of the working

period (i.e. the shift span) of employee r on day d . By definition, their domain is restricted to $[\sigma(r,d), \varepsilon(r,d)]$, thus they are already constrained to lie within the availability window of employee r on day d .

- $\mathcal{A}_{r,d} \subseteq \mathcal{A}^r$

This set represents the activities assigned to employee r on day d . By definition, it is a subset of the activities that the employee is qualified to perform.

- $\mathcal{A}^u \subseteq \mathcal{A}$

This set represents the subset of activities that remain unassigned.

Table 4.2: Summary of Decision variables

Primary Variables	Definition
For every activity $a \in \mathcal{A}$ for every employee $r \in \mathcal{R}$:	
$x_{r,a} \in \{0,1\}$	if activity a is assigned to employee r
$S_a \in \mathcal{T}$	actual start timeslot of activity a
$E_a \in \mathcal{T}$	actual end timeslot of activity a
Secondary Variables	Definition
For every employee $r \in \mathcal{R}$ and every day $d \in \mathcal{D}$:	
$b_{r,d} \in \mathcal{T}$	start timeslot of the shift of employee r on day d
$e_{r,d} \in \mathcal{T}$	end timeslot of the shift of employee r on day d
$\mathcal{A}_{r,d} \subseteq \mathcal{A}^r$	set of activities assigned to employee r on day d
$\mathcal{A}^u \subseteq \mathcal{A}$	unassigned activities

4.3 Requirements

The constraints are organized into three categories: consistency, legal, and company specific constraints, following the structure of the MILP formulation [2]. They are expressed using Constraint Programming specific abstractions and global constraints, replacing the linear formulations of the MILP model.

4.3.1 Consistency constraints

These constraints ensure the internal coherence of the model by linking the decision variables and enforcing the correct logical relationships among them. In particular, they define how auxiliary variables are derived from the primary decision variables.

- **Derived set $\mathcal{A}_{r,d}$**

For notational convenience, we define the set of activities assigned to employee r on day d as

$$\mathcal{A}_{r,d} = \{a \in \mathcal{A}^r \mid x_{r,a} = 1 \wedge \text{day}(a) = d\}, \quad \forall r \in \mathcal{R}, d \in \mathcal{D}. \quad (4.1)$$

This set is fully determined by the assignment variables and is used to simplify the formulation of subsequent constraints.

- **Definition of $\mathcal{A}^u$**

This set represents the activities that are not assigned to any employee and, therefore, whose demand remains unmet. It can be defined as

$$\mathcal{A}^u = \mathcal{A} \setminus \left(\bigcup_{r \in \mathcal{R}} \bigcup_{d \in \mathcal{D}} \mathcal{A}_{r,d} \right). \quad (4.2)$$

- **Definition of $z_{r,d}$**

Employee r is considered to be working on day d if and only if at least one activity is assigned to them on that day. Formally, $\forall r \in \mathcal{R}$ and $\forall d \in \mathcal{D}$:

$$x_{r,a} = 1 \Rightarrow z_{r,d} = 1 \quad \forall a \in \mathcal{A} \text{ such that } \text{day}(a) = d, \quad (4.3)$$

$$z_{r,d} = 1 \Rightarrow \sum_{\substack{a \in \mathcal{A}: \\ \text{day}(a) = d}} x_{r,a} \geq 1. \quad (4.4)$$

In the CP-SAT implementation, this equivalence is enforced using a boolean OR constraint conditioned on $z_{r,d}$:

```
model.add_bool_or(daily_assignments[r,d]).only_enforce_if(
    is_working[r, d])
```

Code Snippet 4.2: When $z_{r,d}$ (implemented as `is_working[r,d]`) is 1, a conditional boolean OR enforces that at least one activity variable $x_{r,a}$ with $\text{day}(a) = d$ (collected in `daily_assignments[r,d]`) is set to 1.

When $z_{r,d} = 1$, the `add_bool_or` constraint requires that the logical OR over the variables in the list evaluates to True, i.e., at least one $x_{r,a}$ with $\text{day}(a) = d$ must be equal to 1. This guarantees that employee r is assigned to at least one activity on day d .

- **Definition of $b_{r,d}$ and $e_{r,d}$**

These variables represent, respectively, the start and end of the working period

of employee r on day d . A standard formulation would enforce the following inequalities:

$$S_a \geq b_{r,d} \quad \forall a \in \mathcal{A}_{r,d}, \quad (4.5)$$

$$E_a \leq e_{r,d} \quad \forall a \in \mathcal{A}_{r,d}. \quad (4.6)$$

This formulation is correct and commonly used in linear models. However, in CP-SAT it is possible to strengthen these constraints by directly defining $b_{r,d}$ and $e_{r,d}$ as the minimum and maximum over the assigned activities. Formally, for all $r \in \mathcal{R}$ and $d \in \mathcal{D}$

$$b_{r,d} = \min\{S_a \mid a \in \mathcal{A}_{r,d}\}, \quad (4.7)$$

$$e_{r,d} = \max\{E_a \mid a \in \mathcal{A}_{r,d}\}. \quad (4.8)$$

In the CP-SAT implementation, this is achieved through the following constraints

```
model.add_min_equality(
    start_shift[r,d],
    [activity_interval[r,a]["start"] for a in daily_activities[r,d]
])

model.add_max_equality(
    end_shift[r,d],
    [activity_interval[r,a]["end"] for a in daily_activities[r,d]]
)
```

Code Snippet 4.3: Definition of shift start and end times in CP-SAT. Both are computed over the activity start/end values in `daily_activities`, the set of activities assigned to employee r on day d .

which enforce that the first argument is equal to the minimum (respectively, maximum) value among the elements of the given list.

- **Non-overlapping activities for each employee**

Each employee $r \in \mathcal{R}$ can be assigned at most one activity at any given time during a day. Formally, for all $r \in \mathcal{R}$, $d \in \mathcal{D}$, and for all distinct activities $a_1, a_2 \in \mathcal{A}_{r,d}$, the following disjunctive constraint must hold:

$$S_{a_1} \geq E_{a_2} \vee S_{a_2} \geq E_{a_1}. \quad (4.9)$$

In the CP-SAT framework, this constraint is efficiently handled through a global `no_overlap` constraint:

```
model.add_no_overlap(daily_intervals[r,d])
```

Code Snippet 4.4: Global CP-SAT no-overlap constraint. `daily_intervals[r,d]` collects all optional interval variables associated with the activities assigned to the employee r on day d , as well as the interval variables representing breaks

This global constraint ensures that no two intervals assigned to the same employee overlap in time, providing a more efficient and compact formulation compared to the pairwise disjunctive encoding.

4.3.2 Legal Requirements

This class of constraints enforces compliance with labor regulations and contractual rules. In particular, they impose limits on working time and rest conditions that must be satisfied by all feasible schedules.

- **Maximum daily working time**

This constraint ensures that the total duration of all activities assigned to an employee on a given day does not exceed their specific daily limit $k_{L1}(r)$: $\forall r \in \mathcal{R}, \forall d \in \mathcal{D}$

$$\sum_{a \in \mathcal{A}_{r,d}} (E_a - S_a) \leq k_{L1}(r). \quad (4.10)$$

Since CP-SAT operates over variable domains, an alternative formulation consists in introducing an auxiliary variable representing the total daily working time of employee r on day d , denoted by $dw_{r,d}$. This variable is defined as:

$$dw_{r,d} = \sum_{a \in \mathcal{A}_{r,d}} (E_a - S_a), \quad (4.11)$$

and its domain can be directly restricted to enforce the constraint:

$$dw_{r,d} \in [0, k_{L1}(r)]. \quad (4.12)$$

This formulation integrates the constraint into the variable definition, allowing CP-SAT to exploit domain propagation more effectively.

- **Maximum weekly working time**

The total amount of work assigned to each employee over the entire planning horizon must not exceed the maximum weekly limit $k_{L2}(r)$: $\forall r \in \mathcal{R}$

$$\sum_{d \in \mathcal{D}} \sum_{a \in \mathcal{A}_{r,d}} (E_a - S_a) \leq k_{L2}(r). \quad (4.13)$$

Analogously to the daily working time constraint, we introduce an auxiliary variable representing the total weekly working time of employee r , denoted by ww_r . This variable is defined as:

$$ww_r = \sum_{d \in \mathcal{D}} dw_{r,d}. \quad (4.14)$$

The weekly constraint can then be enforced by restricting its domain as:

$$ww_r \in [0, k_{L2}(r)]. \quad (4.15)$$

This formulation allows CP-SAT to exploit domain propagation at a higher aggregation level compared to the explicit summation over all days and activities.

- **Maximum consecutive working days**

Each employee $r \in \mathcal{R}$ must have at least one rest day after working at most $k_{L3}(r)$ consecutive days. This constraint also accounts for the number of consecutive working days accumulated before the beginning of the planning horizon, denoted by $w(r)$.

When $w(r) > 0$, the following constraint applies to the initial window:

$$\sum_{d=0}^{k_{L3}(r)-w(r)-1} z_{r,d} \leq k_{L3}(r) - w(r), \quad \forall r \in \mathcal{R}. \quad (4.16)$$

For all subsequent windows within the planning horizon, we enforce:

$$\sum_{d=i}^{i+k_{L3}(r)-1} z_{r,d} \leq k_{L3}(r), \quad \forall r \in \mathcal{R}, \quad \forall i \in \{0, \dots, |\mathcal{D}| - k_{L3}(r)\}. \quad (4.17)$$

- **Maximum daily span**

This constraint differs from the maximum daily working time, since it accounts for the entire time interval in which an employee is present at work, including breaks or periods without assigned activities. Formally, $\forall r \in \mathcal{R}$ and $\forall d \in \mathcal{D}$:

$$e_{r,d} - b_{r,d} \leq k_{L5}(r). \quad (4.18)$$

- **Minimum rest between two working days**

Each employee $r \in \mathcal{R}$ must respect a minimum rest period of at least $k_{L6}(r)$ timeslots between the end of one working period and the start of the next. The variable $h(r) \in \{0, \dots, 95\}$ denotes the last working timeslot of employee r on the day preceding the planning horizon.

For the first day of the horizon, the constraint is defined as:

$$z_{r,0} = 1 \Rightarrow b_{r,0} + (96 - h(r)) \geq k_{L6}(r), \quad \forall r \in \mathcal{R}. \quad (4.19)$$

For all subsequent pairs of consecutive days d and $d + 1$, we enforce:

$$z_{r,d} = 1 \wedge z_{r,d+1} = 1 \Rightarrow b_{r,d+1} - e_{r,d} \geq k_{L6}(r), \quad \forall r \in \mathcal{R}, d \in \mathcal{D} \setminus \{D\}. \quad (4.20)$$

In the CP-SAT implementation, these logical implications are handled using the `only_enforce_if()` method, ensuring that the constraints are activated only when the corresponding working-day conditions are satisfied.

4.3.3 Company regulations

- **Minimum consecutive working time on the same activity**

This constraint enforces a minimum duration requirement for each activity. In general, it can be expressed as:

$$E_a - S_a \geq k_{G3}(a), \quad \forall a \in \mathcal{A}. \quad (4.21)$$

However, in our specific setting, this condition is inherently satisfied by construction, since each activity in the model corresponds to a predefined time block obtained from the decomposition of larger tasks. As a consequence, the duration of each activity is already equal to $k_{G3}(a)$, and no additional constraint is required.

- **Demand satisfaction and assignment exclusivity**

In the MILP formulation, demand satisfaction is typically enforced by requiring that the sum of the contributions of different employees assigned to the same activity, possibly plus a slack variable, covers the total demand.

In our formulation, this aspect is handled differently. Since activities are already preprocessed and decomposed into indivisible units, each activity must be either fully assigned or left unassigned. As a consequence, demand satisfaction reduces to checking whether a belongs to $\mathcal{A}^u$ or not.

However, we must ensure that each activity is assigned to at most one employee. Formally, for all distinct employees $r_1, r_2 \in \mathcal{R}$ and $\forall d \in \mathcal{D}$:

$$\mathcal{A}_{r_1,d} \cap \mathcal{A}_{r_2,d} = \emptyset. \quad (4.22)$$

This constraint is handled in CP-SAT through the `add_at_most_one` constraint, which ensures that not more than one element of the list passed as argument is True

```
model.add_at_most_one([assignment[r,a] for r in employees])
```

Code Snippet 4.5: Enforces assignment exclusivity for each activity a , where $\text{assignment}[r, a]$ corresponds to $x_{r,a}$ and the list includes all employees considered for that activity.

4.3.4 Special handling of the breaks

The modeling of breaks represents one of the main differences between the MILP and the CP formulations.

In the MILP model, breaks are enforced through constraints on the effective working time. In particular, a break is required after a maximum amount of consecutive working time, and a minimum amount of work must be performed after each break.

In the CP formulation, instead, we adopt a simplified but more flexible approach. Breaks are modeled explicitly as optional interval variables, similarly to activities. This allows us to integrate them naturally into the scheduling structure and to handle them through global constraints such as `no_overlap`.

Parameters

- k_{L4} : maximum allowed time without a break (measured on the shift span),
- k_{G1} : minimum working time between consecutive breaks,
- $\text{dur}(b)$: fixed duration of each break.

Decision variables

For each employee $r \in \mathcal{R}$, day $d \in \mathcal{D}$, and break index $i = 1, \dots, n_{r,d}$ (number of breaks of employee r on day d), we introduce:

- $y_{r,d,i} \in \{0, 1\}$: presence of break i .
- $\beta_{s_{r,d,i}}, \beta_{e_{r,d,i}} \in \mathcal{T}$: start and end times of break i with fixed duration:

$$\beta_{e_{r,d,i}} = \beta_{s_{r,d,i}} + \text{dur}(b). \quad (4.23)$$

Each break is modeled as an optional interval variable, which is active only if $y_{r,d,i} = 1$.

Break activation

The number of breaks depends on the length of the working span. Intuitively, longer shifts require more breaks. This is enforced by linking the presence variables to the shift duration:

$$e_{r,d} - b_{r,d} \geq k_{L4} + (i - 1) \cdot k_{G1} \Rightarrow y_{r,d,i} = 1. \quad (4.24)$$

This constraint ensures that the i -th break is activated only if the working span is sufficiently long.

Break positioning

The first break must occur neither too early nor too late within the shift:

$$\beta s_{r,d,1} - b_{r,d} \geq k_{G1}, \quad (4.25)$$

$$\beta s_{r,d,1} - b_{r,d} \leq k_{L4}. \quad (4.26)$$

Similarly, consecutive breaks must be sufficiently spaced:

$$\beta s_{r,d,i+1} - \beta e_{r,d,i} \geq k_{G1}, \quad \forall i, \quad (4.27)$$

$$\beta s_{r,d,i+1} - \beta e_{r,d,i} \leq k_{L4}, \quad \forall i. \quad (4.28)$$

Finally, the last break must not be too close to the end of the shift:

$$e_{r,d} - \beta e_{r,d,n_{r,d}} \geq k_{G1}, \quad (4.29)$$

$$e_{r,d} - \beta e_{r,d,n_{r,d}} \leq k_{L4}. \quad (4.30)$$

Integration with scheduling

Break intervals are included in the set of intervals associated with each employee and day. Therefore, the global `no_overlap` constraint automatically ensures that breaks do not overlap with activities.

This formulation provides a flexible approximation of the original working-time-based rules, while remaining compatible with the interval-based structure of the CP-SAT model.

4.4 Objective function

The objective is to minimize the total cost associated with unmet demand while also accounting for employee preferences. Let f_1 be the penalty function for unassigned activities, defined as:

$$f_1 = \sum_{a \in \mathcal{A}^u} dem(a) \cdot p(a) \quad (4.31)$$

where $\mathcal{A}^u \subseteq \mathcal{A}$ represents the subset of activities that remain unassigned.

Let f_2 be the cost induced by assigning activities to employees according to their preference levels. Since higher preference scores correspond to lower desirability, the function is formulated to penalize unfavorable assignments:

$$f_2 = \sum_{r \in \mathcal{R}} \sum_{d \in \mathcal{D}} \sum_{a \in \mathcal{A}_{r,d}} (E_a - S_a) \cdot c(r, a) \quad (4.32)$$

These are the two primary objectives requested by the company. In our implementation, we adopt a lexicographic optimization approach, as a modeling choice, which is defined and discussed in detail in Chapter 5.

4.5 Adaptation to fixed shifts

Before developing the flexible-shift formulation presented in this chapter, we implemented a simplified version of the model based on predefined shifts. This approach was used as an initial step due to its simpler structure and easier validation.

In this setting, the set of possible shifts is given as input. Let $\mathcal{S} = \{0, \dots, S\}$ denote the set of all shifts. Each shift $s \in \mathcal{S}$ is characterized by:

- $b(s)$: starting time of the shift.
- $e(s)$: ending time of the shift.
- $dur(s) = e(s) - b(s)$: duration of the shift.
- a time window within which the break must be scheduled (we considered only one break per employee for simplicity).

During the preprocessing phase, employee availability is matched with shift intervals in order to define, for each employee $r \in \mathcal{R}$, the subset $\mathcal{S}^r \subseteq \mathcal{S}$ of feasible shifts.

Decision variables

The main difference with respect to the flexible-shift formulation lies in the representation of working periods. Instead of modeling the start and end of each shift through continuous variables $b_{r,d}$ and $e_{r,d}$, we introduce binary variables:

$$j_{r,d,s} \in \{0, 1\}, \quad \forall r \in \mathcal{R}, d \in \mathcal{D}, s \in \mathcal{S}^r, \quad (4.33)$$

where $j_{r,d,s} = 1$ if employee r is assigned to shift s on day d .

To ensure consistency, each employee can be assigned to at most one shift per day:

$$\sum_{s \in \mathcal{S}^r} j_{r,d,s} \leq 1, \quad \forall r \in \mathcal{R}, d \in \mathcal{D}. \quad (4.34)$$

Adaptation of constraints

All constraints involving the shift boundaries are adapted by replacing the decision variables $b_{r,d}$ and $e_{r,d}$ with the corresponding parameters of the selected shift.

In particular:

- *Activity scheduling*: an activity assigned to employee r on day d must lie within the selected shift:

$$j_{r,d,s} = 1 \Rightarrow b(s) \leq S_a \leq E_a \leq e(s), \quad (4.35)$$

for all $a \in \mathcal{A}_{r,d}$.

- *Maximum daily span*: this constraint is implicitly satisfied, since the shift duration $dur(s)$ is fixed and known. In particular, this is enforced during the preprocessing phase, when defining $\mathcal{S}^r$ as the set of feasible shifts for employee r , which already accounts for the maximum allowable daily span.
- *Minimum rest between shifts*: the rest constraint is expressed using shift boundaries:

$$j_{r,d,s} = 1 \wedge j_{r,d+1,s'} = 1 \Rightarrow b(s') - e(s) \geq k_{L6}(r). \quad (4.36)$$

Discussion

The fixed-shift formulation significantly reduces the complexity of the scheduling problem, since working periods are selected from a predefined set rather than constructed dynamically. However, this simplification comes at the cost of reduced flexibility, as it does not allow for personalized schedules tailored to the assigned activities.

For this reason, while the fixed-shift model proved useful during the development phase, the flexible-shift formulation is better suited to capture the requirements of the real-world problem considered in this thesis.

Chapter 5

Lexicographic solution

When multiple objectives are combined within a single optimization model, properly aggregating them becomes a key issue. If the objectives share the same unit of measurement, for instance when they all represent costs or profits, they can be directly summed, and the resulting expression can be minimized or maximized accordingly.

However, when the objectives capture different aspects of the problem, combining them requires additional care. In such cases, a direct aggregation may lead to arbitrary trade-offs, making it difficult to interpret the relative importance of each component without further assumptions or scaling.

In our case, for instance, employee preferences are expressed as a qualitative or ordinal measure of desirability, while penalties associated with unmet demand reflect relative priorities rather than a physical quantity. Nevertheless, both components can be mapped onto a numerical scale, and normalization techniques can be applied to make them comparable within a unified objective function.

A common approach in such situations is to combine the different objectives through a weighted sum, assigning coefficients that reflect their relative importance. In practical applications, such as industrial optimization systems, these weights are typically defined by domain experts or calibrated using historical data and business requirements. The choice of weights plays a crucial role, as it directly influences the resulting trade-offs between objectives.

An alternative approach is lexicographic optimization, which is particularly suitable when a clear priority structure among objectives is known a priori. By definition, this method consists in establishing a strict hierarchy among the objectives and optimizing them sequentially according to their priority.

5.1 Objective Functions and Idle Time

To formalize the lexicographic approach, let x denote a solution vector containing all decision variables of the model (assignment, start, and end times of the activities). We consider the primary objective f_1 (unmet demand penalty) and the secondary objective f_2 (employee preferences cost) as defined in Section 4.4. It is reasonable to assume that assigning an activity to an employee who does not like it is always preferable to leaving the activity demand unmet. Therefore, f_1 holds the absolute highest priority.

To further improve the quality of the solution, we introduce a third objective, f_3 , to minimize the idle time. By idle time, we refer to the time an employee spends at work without being assigned to any activity (excluding mandatory breaks). This is computed as the difference between the daily shift span and the effective time spent working on tasks or on breaks:

$$f_3 = \sum_{r \in \mathcal{R}} \sum_{d \in \mathcal{D}} \left((e_{r,d} - b_{r,d}) - \sum_{a \in \mathcal{A}_{r,d}} (E_a - S_a) - \sum_{i=1}^{n_{r,d}} y_{r,d,i} \cdot \text{dur}(b) \right) \quad (5.1)$$

This additional objective is motivated by three main considerations:

- To reduce the gap between the proposed model and the original problem formulation regarding break management. In the MILP model, breaks are defined based on the time effectively spent on activities, whereas in our CP formulation they are regulated through the overall working span. By minimizing idle time, we reduce the discrepancy between these two measures, bringing the CP model closer to the original formulation.
- To encourage more compact and realistic schedules. In the current formulation, if multiple activities share a wide availability window (e.g., spanning the entire day), the solver has no incentive to schedule them consecutively. As a result, it may produce solutions where activities are scattered throughout the day, even though alternative schedules with consecutive assignments would have the same objective value. From a practical standpoint, however, it is clearly preferable for an employee to work a continuous block of a few hours rather than being required to work at multiple disjoint times during the day.
- To account for efficiency considerations that are not explicitly captured by the current objectives. At present, the model only considers unmet demand and employee preferences; however, in a more realistic setting, one could also introduce explicit labor costs. From this perspective, it is clearly inefficient to assign an employee to a full shift while utilizing only a small portion of their available working time (e.g., assigning three hours of work within an eight-hour shift). Minimizing idle time helps mitigate this inefficiency by promoting a better utilization of the workforce.

It is important to emphasize that the introduction of the third objective f_3 extends the original problem formulation considered in Chapter 4. This objective is not part of the MILP model used by Optit and does not arise from explicit business requirements. Instead, it is introduced in this work as an additional modeling choice aimed at improving solution quality from a scheduling perspective, by promoting more compact and operationally efficient schedules. For this reason, f_3 is not considered in the final evaluation of solutions. The comparison with the MILP model is therefore carried out consistently with the original industrial formulation, i.e., with respect to unmet demand and employee preference satisfaction only (f_1 and f_2).

5.2 The Lexicographic Optimization Sequence

Let Ω be the set of all feasible solutions that satisfy the consistency, legal, and company constraints described in Chapter 4. Given the priority hierarchy $f_1 \gg f_2 \gg f_3$, the model is solved through a sequence of three optimization phases. In each phase, we optimize a single objective while enforcing the optimal values found in the previous steps as hard constraints.

Phase 1: Minimizing Unmet Demand

The solver first addresses the most critical aspect of the problem, finding a feasible schedule that minimizes the unmet demand penalty.

$$z_1^* = \min\{f_1 | x \in \Omega\}$$

Phase 2: Minimizing Preference Cost

Once the optimal penalty z_1^* is found, the solution space is restricted. We define a new feasible space $\Omega_1 = \Omega \cap \{x | f_1 \leq z_1^*\}$. The model is then solved to optimize employee preferences:

$$z_2^* = \min\{f_2 | x \in \Omega_1\}$$

Phase 3: Minimizing Idle Time

Finally, keeping both unmet demand and preference costs at their respective optimal values, we define $\Omega_2 = \Omega_1 \cap \{x | f_2 \leq z_2^*\}$. The last execution compacts the schedules:

$$z_3^* = \min\{f_3 | x \in \Omega_2\}$$

Remark: In some lexicographic optimization schemes, the constraint enforcing the optimal value of a higher-priority objective is relaxed by introducing a tolerance. For instance, one may consider a constraint of the form $\{x | f_1 \leq z_1^* \cdot (1 + \text{toll})\}$, allowing the first objective to slightly deteriorate if this leads to a significant improvement in subsequent objectives.

In our case, this relaxation was not applied in Phase 2, as empirical results showed no improvement in solution quality. It was also not introduced in Phase 3, since the idle

time objective does not contribute to the final evaluation metric and is only used to improve the quality of the solution.

5.3 Implementation details: Warm Starting in CP-SAT

From a computational perspective, building and solving three distinct optimization models from scratch can be highly inefficient. However, the CP-SAT solver provides features that allow an efficient implementation of this framework.

Instead of defining a new model for each phase, the same model object is reused. After Phase 1 is completed, an upper bound equal to z_1^* is dynamically added to the constraint pool. The objective function is then updated to minimize f_2 .

Crucially, to accelerate the search process in Phase 2 and Phase 3, the optimal variable assignments found in the previous phase are passed to the solver as hints. By providing this warm start, the solver's initial search heuristics are guided toward a solution that is already feasible for the restricted space Ω_i . This allows the algorithm to instantly satisfy the new hard constraints before attempting to improve the new objective, significantly reducing computation time and making the lexicographic approach computationally viable.

Chapter 6

Computational results and comparison

In this chapter, we present the implementation of the Constraint Programming model and analyze its computational performance. The obtained results are then compared with those of the MILP formulation introduced in Chapter 3.

Although both models address the same underlying scheduling problem, the comparison is not perfectly equivalent from a methodological perspective. In particular, the two approaches differ both in the management of breaks and in the optimization strategy. The MILP formulation models breaks through constraints on effective working time and optimizes a single weighted sum objective function, whereas the CP formulation explicitly schedules breaks according to shift duration and adopts a lexicographic optimization approach. Consequently, the comparison should be interpreted as an empirical evaluation of two different modeling and solution paradigms applied to two variants of the same operational problem, rather than as a strict one-to-one benchmark.

The discussion of the observed computational behavior is also informed by insights from existing literature on CP and MILP optimization ([5] and [6]). The proposed interpretations should be regarded as exploratory rather than definitive conclusions.

6.1 Implementation of the CP Model

The CP model was implemented in Python using Google OR-Tools and the CP-SAT solver, a hybrid optimization engine combining Constraint Programming, SAT solving, and Integer Programming techniques.

The mathematical formulation introduced in Chapter 4 was directly translated into the CP-SAT framework, with a focus on preserving its structure while adapting it to solver-specific modeling constructs. Decision variables, constraints, and the objective function were implemented using native CP-SAT features, with particular emphasis on modeling choices that support efficient propagation. In particular, interval variables and global constraints were extensively used to represent scheduling decisions and enforce tempo-

ral feasibility in a compact and computationally effective way.

The overall optimization procedure follows the lexicographic scheme described in Chapter 5. From an implementation perspective, the same base model is reused across phases, with only the objective definition being updated between successive runs. To improve solver efficiency, solutions obtained in earlier phases were provided as hints in subsequent ones, allowing CP-SAT to initialize the search from feasible points that already satisfy higher-priority objectives.

All experiments were performed on a MacBook Air equipped with an Apple M4 chip (10 cores: 4 performance and 6 efficiency cores) and 16 GB of RAM, running macOS. A global time limit of 60 minutes was imposed for each instance due to operational requirements. This 60 minutes budget was allocated to the objectives relevant for the industrial evaluation (unmet demand and preferences). Idle-time minimization was then performed separately for analysis purposes, with an additional limit of at most 15 minutes.

6.2 Experimental Setup

The computational analysis was conducted on three problem instances of increasing size. Although the number of instances is limited and therefore does not allow for statistically significant conclusions, the experiments aim to provide preliminary insights into the behavior and scalability of the considered approaches.

The instances are referred to as *Small*, *Medium*, and *Big*, and are characterized as follows:

- **Small:** 15 employees, 43 distinct activities, and 472 total activity requests;
- **Medium:** 66 employees, 67 distinct activities, and 2676 total activity requests;
- **Big:** 88 employees, 290 distinct activities, and 2319 total activity requests.

All of them consider a planning horizon of one week and use a granularity of 15 minutes. Each experiment was repeated 10 times, and the reported results correspond to the average over these runs. This choice was motivated by the presence of randomized components in the CP-SAT search heuristics, which may lead to variability in execution times and intermediate solutions across different executions.

The primary objective is the minimization of unmet demand, followed by the minimization of employee preference costs. In addition, idle time minimization is considered as a further quality indicator aimed at promoting more compact and operationally efficient schedules.

The impact of idle time minimization is discussed in the analysis of the CP-SAT results, since it provides useful insights into the practical quality of the generated schedules.

However, it is not part of the original industrial formulation provided by the company and does not appear in the MILP objective function. Consequently, it is excluded from the comparison between the two approaches, which is performed only with respect to unmet demand and preference costs.

6.3 Results of the CP Model

In this section, we present the computational results obtained with the CP model on the three instances introduced in the previous section. For each instance, we analyze the solver's behavior in terms of solution quality, convergence dynamics, computational time, and ability to satisfy the different objectives of the problem.

Particular attention is devoted to the evolution of the primary objective, namely unmet demand, since it represents both the most critical operational requirement and the first objective in the lexicographic optimization hierarchy. We also discuss the behavior of the secondary objectives, preference satisfaction and idle time reduction.

For each instance, convergence plots are reported to illustrate the evolution of the best solution and the corresponding best bound over time.

6.3.1 Small Instance

For the Small instance, the CP model exhibits very strong performance. The solver is able to find an optimal solution for the primary objective, namely unmet demand, in less than 10 seconds, successfully assigning all activities to the available employees.

Figure 6.1 illustrates the convergence behavior of the CP-SAT solver for the unmet demand objective. The incumbent solution improves rapidly during the first seconds of execution, and the optimum is reached within a very limited computational time.

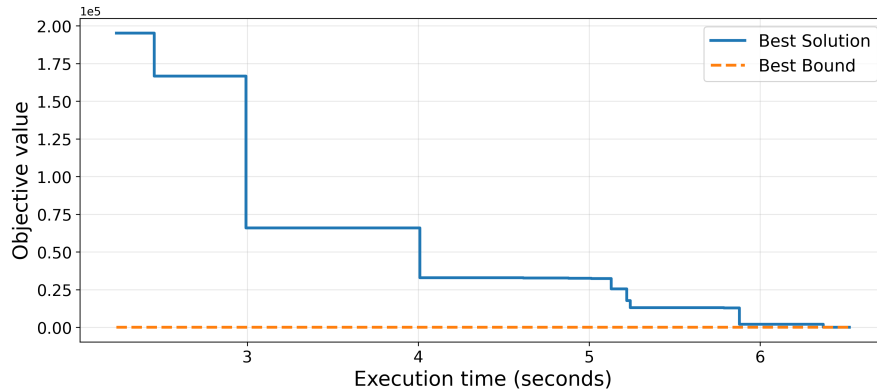


Figure 6.1: Evolution of the best solution and best bound for the unmet demand objective in the Small instance.

After fixing the optimal value of the first objective, the solver proceeds to optimize employee preferences. Preferences are defined on a scale from 0 (most preferred activity) to 7 (least preferred one). The second optimization phase requires on average 140.4 seconds, with a standard deviation of 41.7 seconds across the 10 runs. Under these conditions, the resulting solution achieves an average preference score of 1.1, indicating a high level of employee satisfaction.

Figure 6.2 shows the convergence behavior during the preference optimization phase. Most of the improvement in the incumbent solution occurs during the early stages of the search, while the remaining computational effort is mainly devoted to tightening the lower bound and proving near-optimality. The final optimality gap remains below 1%.

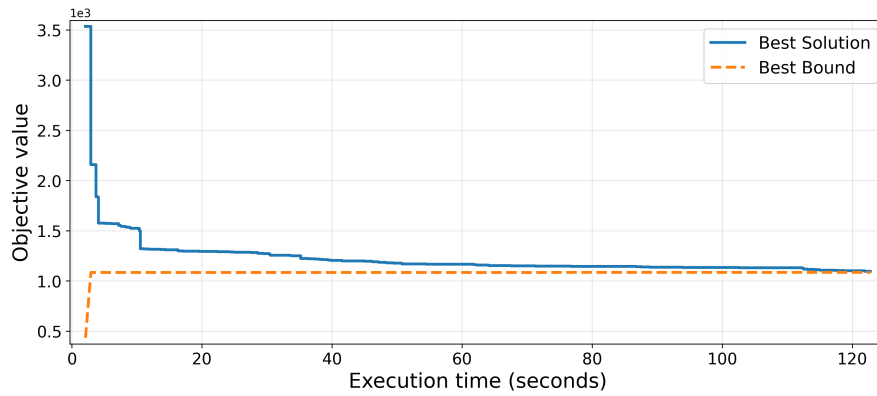


Figure 6.2: Evolution of the best solution and best bound for the preference optimization phase in the Small instance.

The model is also able to almost completely eliminate idle time in approximately 10 minutes, leading to highly compact schedules and almost fully saturated working shifts. From an operational perspective, this produces efficient schedules with very limited inactive periods between activities.

Figure 6.3 illustrates the convergence behavior of the idle time minimization phase. The total idle time decreases sharply during the first minutes of execution, after which only marginal improvements are observed. Although the solver does not formally close the optimality gap during this phase, the obtained schedules already exhibit a shift saturation above 99%, corresponding to only a very limited number of idle slots overall.

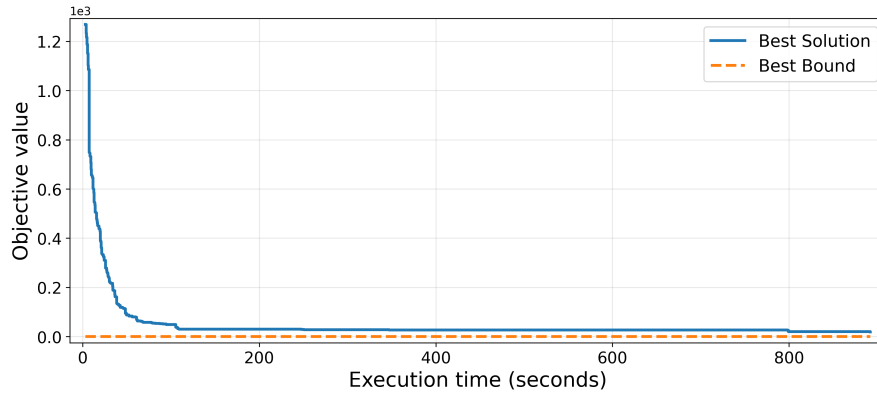


Figure 6.3: Evolution of the best solution and best bound for the idle time objective in the Small instance.

Overall, the CP model is able to compute solutions of very high quality within a limited computational time, while maintaining excellent practical scheduling performance.

6.3.2 Medium Instance

In the Medium instance, all preference values provided in the original industrial dataset were equal to zero. This means that no employee-activity preference structure was specified for this scenario. Consequently, the problem effectively reduces to the minimization of unmet demand, while idle time is considered only as an additional quality indicator.

Also in this case, the CP model is able to assign all activities to employees and reach the optimal value for the primary objective. Over the 10 executions, the optimization of unmet demand required on average 320.3 seconds, with a standard deviation of 51.8 seconds.

Figure 6.4 illustrates the convergence behavior of the CP-SAT solver for the unmet demand objective. The search initially progresses gradually, with several substantial improvements during the first half of the execution. Around 120 seconds, the incumbent solution is already very close to zero unmet demand, after which the solver performs smaller refinements until proving optimality at approximately 320 seconds.

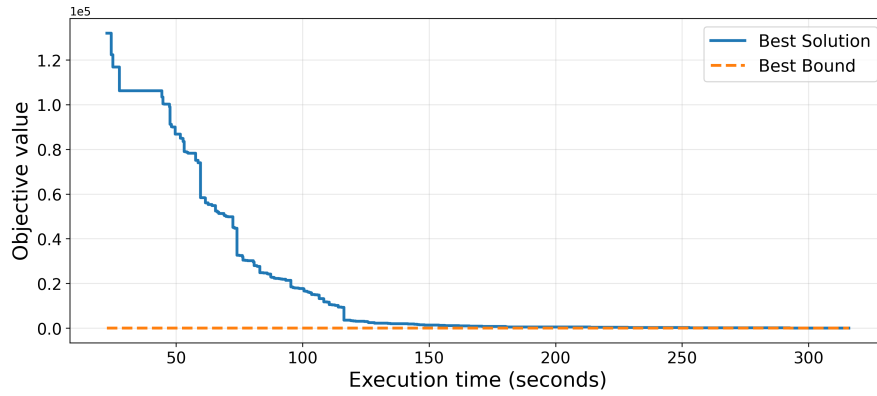


Figure 6.4: Evolution of the best solution and best bound for the unmet demand objective in the Medium instance.

Since preference costs are null for all assignments, the second phase of the lexicographic optimization process is effectively skipped.

After achieving full assignment, a time budget of 15 minutes was allocated to the minimization of idle time. During this phase, the solver is able to almost completely eliminate idle periods between consecutive activities, producing highly compact schedules. This leads to an average shift saturation above 98%, indicating that employees remain occupied for nearly the entire duration of their shifts.

Figure 6.5 shows the convergence behavior during the idle time minimization phase. The total idle time decreases steadily throughout the search, with most improvements occurring during the first part of the execution. Although the optimality gap is not closed within the allocated time, the obtained schedules already exhibit a very limited amount of idle time and a high level of operational compactness.

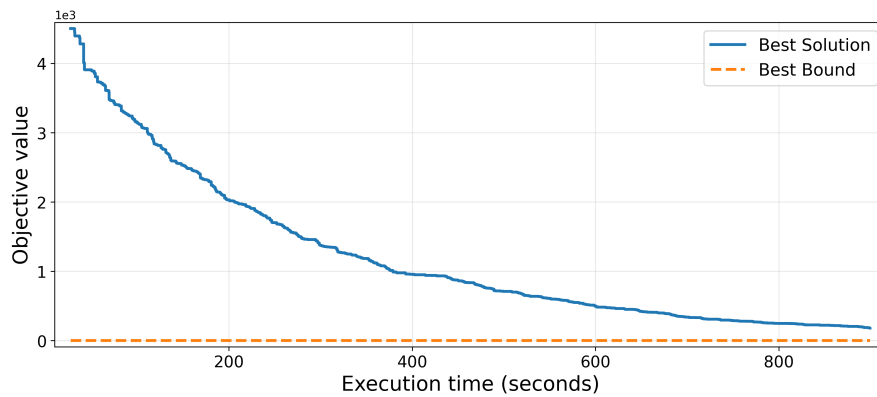


Figure 6.5: Evolution of the best solution and best bound for the idle time objective in the Medium instance.

6.3.3 Big Instance

The Big instance represents the most challenging scenario for the CP model due to its larger size and increased combinatorial complexity. Nevertheless, the solver is still able to compute high-quality solutions within the imposed operational time limit.

For this instance, the available computational budget associated with unmet demand and preference optimization was equally divided between the first two phases of the lexicographic procedure. While the unmet demand optimization consistently reaches the target relative optimality gap before the allocated 30 minutes, the full budget was preserved for all experiments to maintain a uniform evaluation framework and because of the variability across runs.

During the first phase, the solver progressively improves the incumbent solution and converges to solutions with an unmet demand cost of approximately 48060, corresponding to more than 95% of the total demand being covered. The final optimality gap remains below 1%, showing that the solver is able to produce solutions very close to optimality even for the largest instance.

Figure 6.6 illustrates the convergence behavior for the unmet demand objective. The objective value decreases steadily throughout the search, with several major improvements occurring between 200 and 850 seconds. After this phase, the incumbent solution stabilizes and only marginal refinements are observed before the stopping criterion is reached.

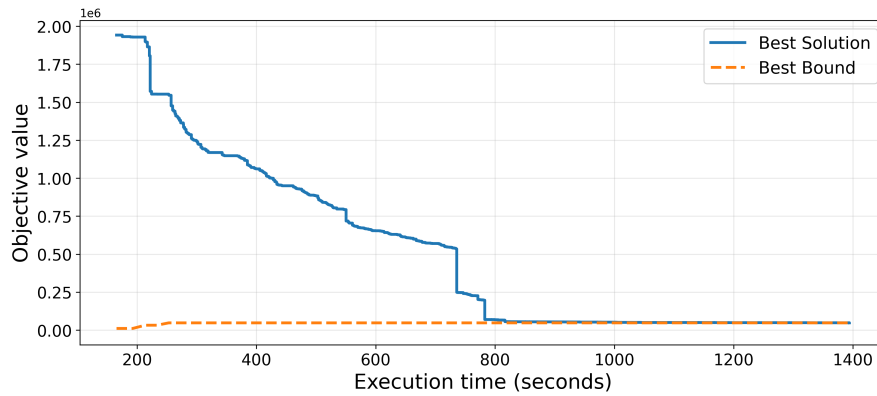


Figure 6.6: Evolution of the best solution and best bound for the unmet demand objective in the Big instance.

After fixing the unmet demand value, the solver proceeds to optimize employee preferences. The schedules obtained after the first phase already exhibit relatively good preference costs, leaving limited room for further improvements.

Figure 6.7 illustrates the convergence behavior during the preference optimization phase. The lower bound improves during the initial stage of the search and then remains es-

entially stable for the remainder of the execution. In contrast, the incumbent solution continues to improve gradually throughout the available time budget, although the rate of improvement decreases over time.

The final optimality gap remains below 3%, showing that the obtained solution is close to optimal despite the size and complexity of the instance.

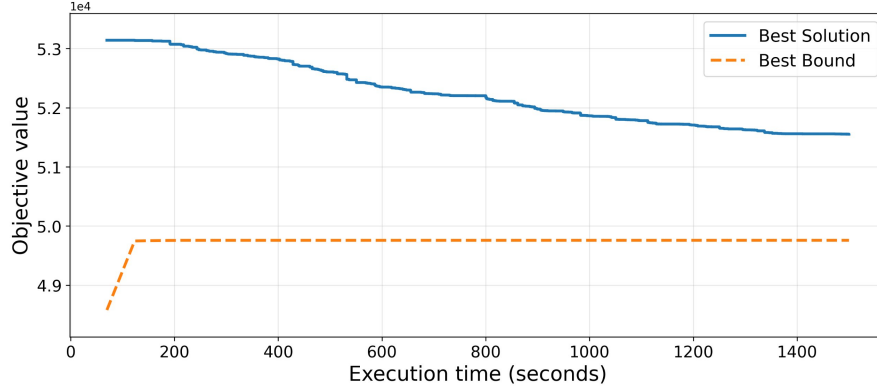


Figure 6.7: Evolution of the best solution and best bound for the preference objective in the Big instance.

Since the entire one-hour operational budget was allocated to the first two objectives, the idle time minimization phase was not included in the reported experiments for this instance. Nevertheless, preliminary tests indicate that allocating even an additional 10 minutes to this phase reduces total idle time by approximately 20%, leading to more compact schedules and higher shift saturation.

Overall, despite the significantly increased complexity of the instance, the CP model remains capable of generating operationally competitive schedules with high demand coverage and good employee satisfaction.

6.4 Comparison with the MILP Model

This section compares the performance of the CP model with the MILP formulation implemented in Optit using the FICO Xpress solver. The MILP experiments were conducted on a machine equipped with 16 GB of RAM and a single processor with 6 cores (12 logical threads).

Since the CP model adopts a lexicographic optimization strategy, its results are reported separately for each optimization phase. In particular, the values associated with unmet demand and preference costs are presented independently. In particular, the reported execution times refer to the specific optimization phase associated with each column. By

contrast, the MILP formulation optimizes a single aggregated objective corresponding to the sum of unmet demand and preference costs.

Idle time is excluded from the comparison, as it is not part of the original industrial formulation and does not appear explicitly in the MILP objective function. Nevertheless, its impact on schedule quality was discussed separately in the analysis of the CP results.

6.4.1 Small Instance

For the Small instance, the MILP formulation reaches a final optimality gap of 23% after the full one-hour time limit, indicating a significantly slower convergence compared to the CP approach.

Table 6.1 summarizes the results obtained by the two approaches. For the CP model, the first column reports the outcome of the optimization phase dedicated to unmet demand, while the second column reports the final results after the optimization of preference costs.

Table 6.1: Results for the Small instance

Metric	CP-SAT (unmet)	CP-SAT (preferences)	MILP (unmet + preferences)
Best Solution	0	1094,3 $\pm$ 1,4	1322
Best Bound	0	1089	1018
Gap	0%	0,48%	23%
Cost from Unmet Demand	0	0	0
Cost from Preferences	3343,9 $\pm$ 173,8	1094,3 $\pm$ 1,4	1322
Execution Time (s)	7.9 $\pm$ 0.8	140.4 $\pm$ 41.7	3600

6.4.2 Medium Instance

This instance shows a marked difference in the behaviour of the two approaches. The CP-SAT solver is able to construct feasible schedules early in the search and progressively improve the incumbent solution in about five minutes.

In contrast, the MILP formulation is unable to produce any feasible integer solution within one hour of computation. The solver progress is limited to the refinement of the continuous relaxation, without entering the integer-feasible region of the search space. This difference suggests that, for this instance size, the combinatorial structure of the problem is handled more effectively by the CP formulation, which is able to exploit constraint propagation to rapidly enforce feasibility, whereas the MILP approach remains dominated by the difficulty of closing the integrality gap.

Table 6.2: Results for the Medium instance

Metric	CP-SAT (unmet)	MILP (unmet)
Best Solution	0	-
Best Bound	0	-
Gap	0%	-
Cost from Unmet Demand	0	-
Execution Time (s)	320.3 ± 51.8	3600

6.4.3 Big Instance

The MILP formulation proves particularly effective in this large-scale setting, reaching a high-quality solution in approximately 7/8 minutes and achieving a final optimality gap of about 1%. This indicates a very strong convergence behavior under the aggregated objective formulation, supported by the strength of the linear relaxation, which provides tight bounds even for large instances.

The CP model, instead, explores the same instance under a lexicographic optimization scheme, producing a feasible solution of high quality but requiring a larger computational budget to refine the different objective levels.

The results are summarized in Table 6.3.

Table 6.3: Results for the Big instance

Metric	CP-SAT (goal: unmet)	CP-SAT (goal: prefs)	MILP (goal: unmet + prefs)
Best Solution	$48060 \pm 63,25$	$51286,8 \pm 99,6$	98387,2
Best Bound	47640	49771,93	97452
Gap	0,87%	2,95%	0,95%
Cost from Unmet Demand	$48060 \pm 63,25$	$48060 \pm 63,25$	48120
Cost from Preferences	$52831,4 \pm 51,1$	$51286,8 \pm 99,6$	50267.2
Execution Time(s)	$1454 \pm 63,3$	1800	431

6.5 Discussion

This section provides a qualitative interpretation of the observed differences between the CP and MILP approaches. The considerations presented here should be regarded as plausible explanations supported by empirical evidence and literature, rather than definitive conclusions.

A first relevant observation is that the CP-SAT approach performs competitively overall, especially considering that it relies on a free and open-source solver, whereas the MILP formulation was solved using a commercial-grade optimization solver. Despite this difference, CP-SAT is able to produce high-quality feasible solutions in all instances, highlighting the strength of modern hybrid CP engines.

A key aspect in interpreting the results is the nature of the primary objective, namely unmet demand. In the Small and Medium instances, this objective admits a natural lower bound equal to zero, which is also achievable. Consequently, these instances behave primarily as feasibility-driven scheduling problems rather than as classical cost-minimization problems.

This setting is particularly favorable to Constraint Programming. Once a feasible solution with zero unmet demand is found, optimality is immediately certified, since no better value exists. CP solvers rely heavily on propagation and primal search, and when feasibility at the optimal bound is reachable, propagation becomes highly effective in pruning the search space [6]. As observed by Naderi et al. [5], this effect is especially strong for summation-type objectives, where propagation weakens significantly once the upper bound becomes strictly positive because the slack gets distributed across the sum terms.

In contrast, the Big instance behaves as a genuine optimization problem, since some amount of unmet demand becomes unavoidable. In this regime, the quality of the dual bounds becomes substantially more important, which appears to favor the MILP formulation. Strong linear relaxations provide effective guidance to the branch-and-bound process and allow MILP solvers to progressively tighten the optimality gap [6]. CP solvers, on the other hand, may still identify excellent feasible solutions through propagation and combinatorial search, but often struggle to provide equally strong lower bounds, leading to larger optimality gaps despite the quality of the incumbent solutions [5].

Another relevant aspect concerns the structural composition of the instances and the balance between assignment decisions and temporal sequencing. In the Small and Medium instances, the ratio between activity requests and employees is very high (approximately 31.5 and 40.5 activities per employee, respectively), creating deeply constrained sequencing problems. In such settings, CP effectively exploits propagation to manage long activity sequences, whereas MILP formulations suffer from weaker relaxations caused by the large number of disjunctive big-M constraints [5].

In the Big instance, however, this dynamic partially reverses. Although the total number of activities remains large, the activities-to-employees ratio is the lowest of the three, 26.4, while the number of employees reaches its maximum value, 88. This enlarges the assignment dimension of the problem. As highlighted in the literature, CP performance may deteriorate in assignment-dominated settings, where propagation struggles to prune large pools of alternative resources, whereas MILP solvers naturally benefit from tighter

assignment relaxations [5].

Therefore, the empirical success of the MILP approach on the Big instance may be also attributed to the underlying structure becoming relatively “wider”, with a larger emphasis on assignment flexibility rather than tightly constrained temporal sequencing. This setting appears to better exploit the strengths of dual bounding and linear relaxations in mathematical programming [6]. Although this interpretation is consistent with the observed computational behavior, additional experiments on larger benchmark sets would be required to validate this hypothesis.

Finally, extending the analysis to a larger and more diverse set of instances would be important to confirm these trends and better understand the regimes in which each modeling paradigm is most effective.

Chapter 7

Conclusion

This thesis investigated the use of Constraint Programming for the Shift-Design Personnel Task Scheduling Problem, a workforce scheduling problem in which employee shifts are generated dynamically while activities are assigned to employees over time. Starting from the industrial MILP formulation currently used in practice, a CP model based on the CP-SAT solver was developed. The proposed formulation exploits interval variables, optional activities, and global scheduling constraints to represent temporal relationships and assignment decisions in a natural way. A lexicographic optimization procedure was also introduced to handle the multiple objectives arising in the application.

The computational experiments show that the CP approach is able to produce high-quality solutions on realistic instances while preserving modeling flexibility. In particular, the interval-based structure of CP-SAT proved effective for expressing complex scheduling constraints such as non-overlapping activities, shift spans, and break placement. At the same time, the experiments highlight that the relative performance of CP and MILP depends on the structure of the instance: MILP remains highly competitive, especially in settings dominated by assignment decisions and broader combinatorial flexibility.

These results suggest that Constraint Programming is a viable alternative for the SDPTSP, especially when the problem involves rich temporal logic and complex scheduling rules. However, the study also confirms that no single optimization paradigm is universally superior; the effectiveness of CP or MILP depends on the specific balance between temporal structure, assignment complexity, and relaxation strength.

Several limitations remain. The CP model relies on preprocessing activities into indivisible blocks, and break management is handled through a simplified approximation based on shift spans rather than effective working time. Furthermore, the computational study was conducted on a small set of test instances, and the observed results may not necessarily generalize.

Future work could therefore explore hybrid CP–MILP approaches, richer break formu-

lations, alternative decomposition strategies, and larger computational studies on additional real-world instances.

Overall, the thesis shows that modern Constraint Programming techniques, and in particular CP-SAT, provide a powerful and expressive framework for complex personnel scheduling problems, while also emphasizing the importance of selecting the optimization paradigm according to the structural characteristics of the application at hand.

Bibliography

- [1] Alexander Bockmayr and John N. Hooker. “Constraint Programming”. In: *Discrete Optimization*. Ed. by K. Aardal, G.L. Nemhauser, and R. Weismantel. Vol. 12. Handbooks in Operations Research and Management Science. Elsevier, 2005, pp. 559–600. DOI: [https://doi.org/10.1016/S0927-0507\(05\)12010-6](https://doi.org/10.1016/S0927-0507(05)12010-6). URL: <https://www.sciencedirect.com/science/article/pii/S0927050705120106>.
- [2] Nicolò Gusmeroli and Andrea Bettinelli. *A Mixed-Integer Linear Program to create the shifts in a supermarket*. 2024. URL: <https://arxiv.org/abs/2403.17850>.
- [3] Klaus H. Ecker Jacek Blazewicz et al. *Handbook on Scheduling*. International Handbooks on Information Systems. Springer, June 2007. URL: <https://ideas.repec.org/b/spr/ihinfo/978-3-540-32220-7.html>.
- [4] Tanguy Lapègue, Odile Bellenguez-Morineau, and Damien Prot. “A constraint-based approach for the shift design personnel task scheduling problem with equity”. In: *Computers & Operations Research* (2013).
- [5] Bahman Naderi, Rubén Ruiz, and Vahid Roshanaei. “Mixed-Integer Programming vs. Constraint Programming for Shop Scheduling Problems: New Results and Outlook”. In: *INFORMS Journal on Computing* (2023). DOI: 10.1287/ijoc.2023.1287. URL: <https://ideas.repec.org/a/inm/orijoc/v35y2023i4p817-843.html>.
- [6] Domenico Salvagnin. “Constraint programming techniques for mixed integer linear programs”. PhD thesis. University of Padova, 2009.
- [7] Jorne Van den Bergh et al. “Personnel scheduling: A literature review”. In: *European Journal of Operational Research* 226.3 (2013), pp. 367–385. ISSN: 0377-2217. DOI: <https://doi.org/10.1016/j.ejor.2012.11.029>. URL: <https://www.sciencedirect.com/science/article/pii/S0377221712008776>.